

ECOTECH SOLUTIONS

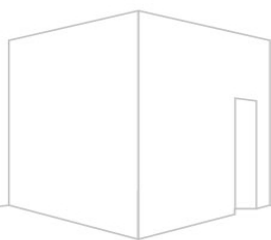
Modelo UML, análisis de POO

Asignatura: Programación Orientada a Objetos Seguros

Sección: TI3V21

Nombre del académico: Rubén Schnettler

Nombre de Estudiante: Sergio Antonio Ruiz Espinoza



Fecha de entrega lunes 07 de septiembre del 2026

Contenido

I. Introducción	3
II. Objetivos	4
Diagnóstico del problema	4
Objetivos del nuevo sistema	4
III. Desarrollo	5
Análisis del problema desde la POO	5
Identificación y jerarquización de entidades	5
Elementos del problema, atributos, objetos y responsabilidades	5
Fundamentos de POO aplicados	6
Implicancias para la seguridad	6
Cómo la POO estructura la solución	6
Diseño del sistema	7
Evolución del diseño	7
Diagrama de clases UML Texto.....	7
Diagrama de clase UML esquema.....	8
Relaciones, multiplicidades y acoplamiento	9
Viabilidad para una implementación en Python	9
Uso de herramientas de IA	9
Primera iteración	9
Segunda iteración	10
Registro del resultado de las dos iteraciones	10
Análisis crítico de cuatro elementos	11
Mejoras aplicadas	11
Principios de diseño POO	11
Matriz de trazabilidad de requerimientos	12
IV. Conclusiones	13
V. Referencias bibliográficas	14

I. Introducción

EcoTech Solutions es una empresa que desarrolla tecnologías sostenibles y ha crecido rápidamente. Ese crecimiento también ha hecho más difícil mantener ordenada la información de empleados, departamentos, proyectos y horas trabajadas. Actualmente, parte de los datos se maneja en planillas y sistemas separados, por lo que pueden aparecer registros duplicados, errores de asignación y diferencias entre la información.

Además del problema de organización, hay datos que necesitan una protección especial, como los salarios y las credenciales de acceso. Por esta razón, la propuesta no se limita a ordenar las clases, sino que también considera el encapsulamiento, la autenticación, la autorización y la validación de datos.

El objetivo de este informe es analizar el caso desde la Programación Orientada a Objetos, construir un modelo de clases UML y revisar la propuesta con el apoyo de una herramienta de Inteligencia Artificial. La decisión final se toma comparando las sugerencias obtenidas con las reglas del caso y con los contenidos de la asignatura.

II. Objetivos

Diagnóstico del problema

Problema	Situación observada
1. Registros duplicados	Una misma persona puede aparecer más de una vez y con datos diferentes.
2. Asignaciones incorrectas	Puede haber errores al asignar empleados a proyectos.
3. Falta de trazabilidad	No siempre es fácil saber quién trabajó, en qué proyecto y en qué fecha.
4. Reportes poco confiables	Si los datos de origen están separados o tienen errores, los reportes también pueden tenerlos.
5. Riesgos de seguridad	Los salarios y las contraseñas necesitan controles para evitar accesos no autorizados.

Objetivos del nuevo sistema

- Mantener un identificador único para cada empleado.
- Controlar que cada empleado pertenezca a un solo departamento a la vez.
- Permitir que un empleado participe en uno o varios proyectos.
- Registrar fecha, horas, descripción, empleado y proyecto en cada registro de tiempo.
- Validar que las horas ingresadas sean mayores que cero y razonables.
- Proteger la información sensible mediante encapsulamiento y almacenamiento seguro.
- **Dejar una estructura que permita generar reportes con información más consistente.**

III. Desarrollo

Análisis del problema desde la POO

Para transferir el problema a un modelo orientado a objetos, primero se separaron las partes del sistema que tienen información y responsabilidades propias. A partir de ello, se definieron las clases y, luego, las relaciones entre ellas.

Identificación y jerarquización de entidades

Nivel	Clase	Rol	Motivo dentro del Sistema
1	Departamento	Entidad principal	Organiza a los empleados y tiene un gerente asociado.
1	Proyecto	Entidad principal	Representa una iniciativa y reúne a los empleados que participan en ella.
2	Empleado	Entidad principal de personal	Es el elemento que se relaciona con departamentos, proyectos y registros de tiempo.
3	RegistroTiempo	Entidad operativa	Une las horas trabajadas con un empleado y un proyecto.
3	Usuario	Control de acceso	Maneja autenticación y autorización para proteger el sistema.

La jerarquía anterior se basa en el nivel de interacción. Departamento y Proyecto son estructuras principales del negocio. Empleado se encuentra en el centro porque participa en ambas. RegistroTiempo depende de un empleado y de un proyecto para tener sentido. Usuario se mantiene separado de Empleado porque su responsabilidad principal es el acceso al sistema y no la administración de los datos laborales.

Elementos del problema, atributos, objetos y responsabilidades

Clase	Atributos principales	Objeto posible	Responsabilidad
Empleado	idEmpleado, nombre, dirección, teléfono, correo, fechaContrato, salarioCifrado, departamento, proyectos	emp_01 = Empleado(nombre='Ana Gómez', ...)	Representar a un trabajador y mantener sus datos laborales.
Departamento	idDepartamento, nombre, gerente, empleados	depto_ti = Departamento(nombre='TI', gerente=emp_01)	Organizar empleados y administrar la estructura del área.
Proyecto	idProyecto, nombre, descripción, fechaInicio, empleados, registrosTiempo	proy_solar = Proyecto(nombre='SolarTech', ...)	Representar un proyecto y administrar a sus participantes.
RegistroTiempo	fecha, horas, descripción, empleado, proyecto	reg_01 = RegistroTiempo(fecha=..., horas=8.0, ...)	Registrar y validar las horas trabajadas.
Usuario	idUsuario, nombreUsuario, contrasenaHash, rol, empleadoAsociado	usr_01 = Usuario(nombreUsuario='ana01', ...)	Controlar el ingreso al sistema y los permisos.

Fundamentos de POO aplicados

En este caso, se pueden aplicar los tres fundamentos trabajados en la unidad: abstracción, encapsulamiento y herencia. No todos se usan de la misma forma, ya que el modelo debe responder al problema y no agregar elementos solo para cumplir con una lista.

Fundamento	Aplicación en el caso	Efecto en el diseño y seguridad
Abstracción	Se representan solo los datos y comportamientos que son necesarios para administrar empleados, departamentos, proyectos, horas y acceso.	Evita guardar información que no aporta al sistema y hace que las clases sean más fáciles de mantener.
Encapsulamiento	Los atributos se manejan como privados y el acceso a información sensible se realiza mediante métodos.	Reduce la posibilidad de modificar directamente datos como el salario o las credenciales.
Herencia	No se crea una jerarquía de empleados porque el caso no presenta tipos de empleados con comportamientos suficientemente diferentes.	Evita una estructura artificial. Si más adelante aparecen especializaciones reales, la herencia podría incorporarse.

Implicancias para la seguridad

Control de acceso: Usuario concentra la autenticación y autorización. Así, no se mezclan las credenciales con la información laboral del empleado.

Protección de datos: El salario se representa como un atributo privado y no se expone directamente. El acceso se realiza mediante un método que puede comprobar permisos.

Validación: RegistroTiempo incorpora validarHoras() para impedir valores menores o iguales a cero y controlar que las horas estén dentro del rango definido.

Cifrado: El atributo salarioCifrado representa conceptualmente información sensible protegida. Sin embargo, el uso de bytes por sí solo no significa que el dato esté cifrado. Además del salario, los datos personales que requieran protección deberán almacenarse de forma segura y, cuando corresponda, mediante mecanismos de cifrado adecuados. En una implementación real también será necesario gestionar correctamente el algoritmo de cifrado, las claves y los controles de acceso a esta información.

Cómo la POO estructura la solución

El enfoque orientado a objetos permite dividir el sistema en partes que tienen una función clara. Por ejemplo, Empleado mantiene la información de la persona, Departamento organiza a los trabajadores, Proyecto administra las iniciativas y RegistroTiempo registra las horas. Usuario queda separado para manejar el acceso. Esta separación facilita el mantenimiento, ya que un cambio en una responsabilidad no obliga a modificar todo el sistema. También deja una base que puede crecer. Por ejemplo, si EcoTech necesita, más adelante, agregar nuevos tipos de proyectos o nuevas reglas de autorización, se pueden ampliar las clases correspondientes sin mezclar esas funciones con el registro de empleados.

Diseño del sistema

Evolución del diseño

Durante la revisión del modelo inicial, se encontraron dos problemas principales. El primero fue la relación entre Departamento y Empleado. El segundo fue que algunos atributos y métodos eran demasiado generales. En la primera versión, la relación Departamento–Empleado se planteó como composición. Después de revisar el caso, se cambió a agregación. La razón es que un empleado no deja de existir si un departamento cambia, se elimina o deja de utilizarse; el empleado puede ser reasignado a otro departamento.

También se mejoraron las firmas de los métodos. En lugar de dejar solamente nombres como registrar(), el modelo final incluye parámetros, tipos de datos y valores de retorno cuando corresponde. Esto hace que el diagrama sea más útil como base para una futura implementación en Python.

Diagrama de clases UML

El modelo final contiene cinco clases. Las líneas continuas representan asociaciones entre las clases del dominio, mientras que la relación Departamento–Empleado usa agregación. La línea discontinua entre Usuario y Empleado representa la asociación de una cuenta de acceso con un empleado y no implica que una clase sea propietaria de la otra.

EcoTech Solutions - Diagrama de Clases UML

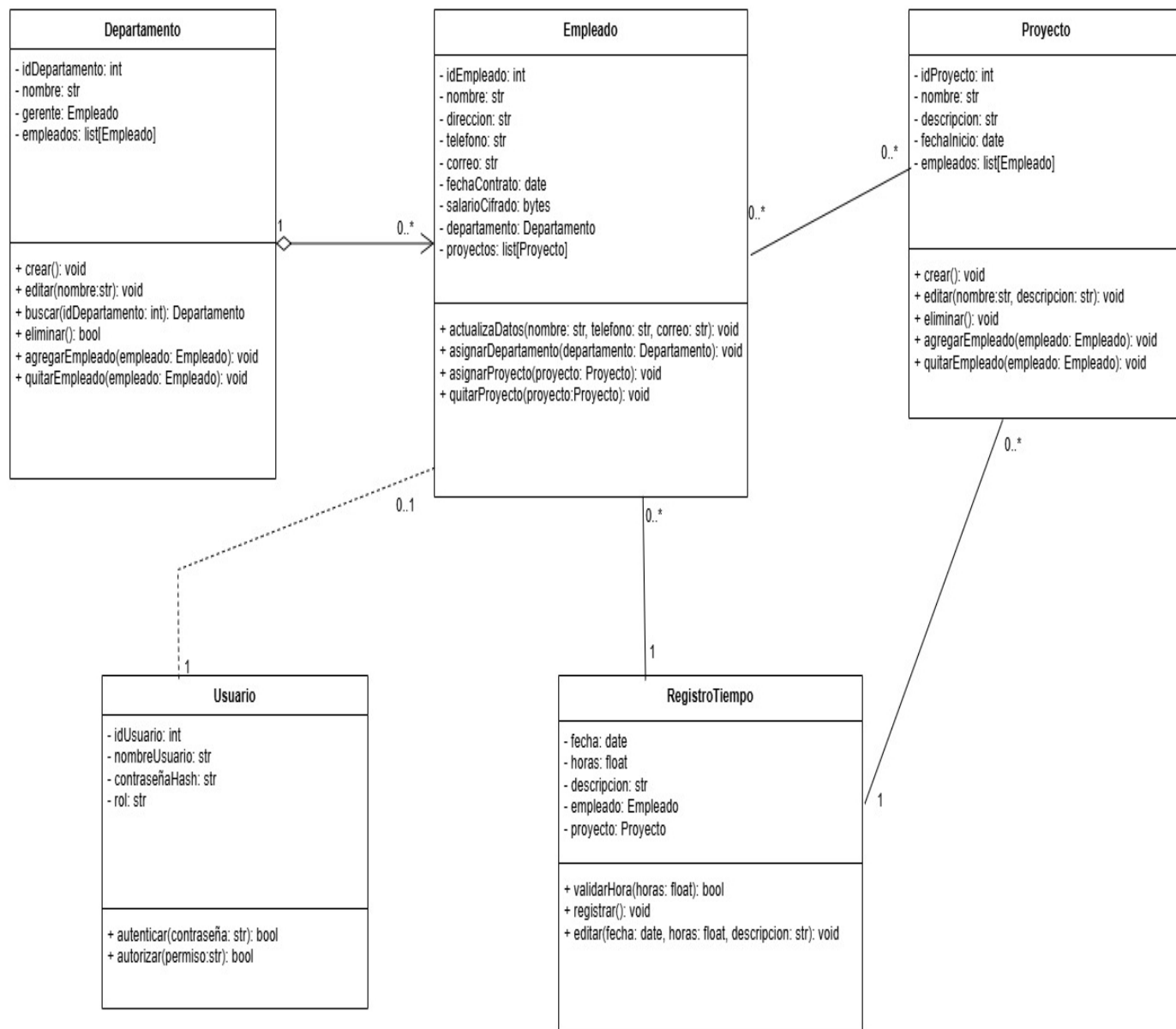


Figura 1: Diagrama de clases UML formalizado para EcoTech Solutions

Relaciones, multiplicidades y acoplamiento

Relación	Tipo	Multiplicidad	Justificación
Departamento – Empleado	Agregación	Departamento 1 / Empleado 0..*	Un departamento puede tener varios empleados. Cada empleado pertenece a un solo departamento a la vez. El empleado puede seguir existiendo si cambia el departamento.
Empleado – Proyecto	Asociación	0..* / 0..*	Un empleado puede participar en varios proyectos y un proyecto puede tener varios empleados.
Empleado – RegistroTiempo	Asociación	Empleado 1 / RegistroTiempo 0..*	Cada registro corresponde a un empleado y un empleado puede tener muchos registros.
Proyecto – RegistroTiempo	Asociación	Proyecto 1 / RegistroTiempo 0..*	Cada registro corresponde a un proyecto y un proyecto puede tener muchos registros.
Usuario – Empleado	Dependencia/asociación de acceso	Usuario 0..1 / Empleado 1	Un empleado puede tener una cuenta de acceso, pero la cuenta no controla el ciclo de vida del empleado.

El modelo busca mantener un acoplamiento razonable. Si Departamento fuera responsable del ciclo de vida de Empleado mediante composición, un cambio en el departamento tendría consecuencias más fuertes. Con agregación, ambos objetos pueden mantenerse de forma independiente. Además, las responsabilidades se separan mediante métodos concretos, por lo que un cambio en la autenticación no debería obligar a cambiar la lógica de registro de empleados.

Viabilidad para una implementación en Python

Las clases propuestas se pueden llevar a Python mediante clases, atributos privados y métodos. Las listas permiten manejar relaciones de uno a muchos y muchos a muchos. Las validaciones se pueden realizar antes de cambiar el estado de un objeto y, cuando corresponda, se pueden lanzar excepciones como ValueError. La persistencia, el cifrado real y la administración de usuarios tendrían que implementarse en una etapa posterior.

Uso de herramientas de IA

Se utilizó una herramienta de inteligencia artificial como apoyo para obtener una primera propuesta y después revisarla. La IA se tomó como una ayuda para comparar alternativas, no como la fuente final de las decisiones.

Primera iteración

Contexto y prompt utilizado:

“Actúa como un arquitecto de software experto en POO. Genera un modelo de clases inicial para resolver el caso de EcoTech Solutions. Necesitamos gestionar empleados, departamentos, proyectos y horas trabajadas. Diseña un diagrama de clases plano, con atributos y nombres de métodos en formato de texto.”

Resultado de la primera iteración: la propuesta incluyó Empleado, Departamento, Proyecto y RegistroTiempo. La relación Departamento–Empleado quedó planteada como composición. Los atributos aparecían con poca protección, los métodos no tenían firmas completas y no se consideró una clase específica para autenticación y autorización.

Qué se tomó y qué se descartó:

Se tomó como base la identificación de las cuatro clases principales y la idea de relacionar RegistroTiempo con Empleado y Proyecto. Se descartó la composición Departamento–Empleado y también se modificó la forma de manejar los datos sensibles.

Segunda iteración

Contexto y prompt utilizado:

“El modelo anterior presenta una relación Departamento–Empleado demasiado fuerte. En el caso planteado, un empleado debe poder seguir existiendo y ser reasignado si cambia o deja de existir un departamento. Refina el modelo considerando una relación de agregación, manteniendo encapsulamiento para los datos sensibles y métodos con parámetros y tipos de retorno.”

Resultado de la segunda iteración: se incorporó la agregación, se reforzó el encapsulamiento, se agregó Usuario para separar autenticación y autorización, y los métodos quedaron definidos con parámetros y tipos de retorno. Esta segunda propuesta se usó como referencia para construir el modelo final.

Registro del resultado de las dos iteraciones

Etapa	Resultado resumido
Iteración 1	4 clases; composición Departamento–Empleado; atributos públicos; métodos sin firmas completas; sin clase de seguridad.
Iteración 2	Agregación Departamento–Empleado; atributos privados; Usuario; métodos con parámetros y retornos.
Modelo final	Se conserva la estructura útil de la segunda iteración, pero se revisan relaciones, multiplicidades y responsabilidades antes de aceptarla.

Análisis crítico de cuatro elementos

Elemento	Propuesta IA	Decisión final	Motivo
1. Departamento–Empleado	La IA propuso composición.	Se cambió a agregación.	El empleado puede seguir existiendo y ser reasignado. No depende del ciclo de vida del departamento.
2. Encapsulamiento	La primera propuesta dejaba los atributos con poca protección.	Se utilizaron atributos privados y métodos públicos.	Los datos sensibles no deberían modificarse directamente desde cualquier parte del programa.
3. Registro de tiempo	La validación no estaba suficientemente separada.	Se agregó validarHoras().	La regla de horas mayores que cero queda dentro de la responsabilidad de RegistroTiempo.
4. Autenticación y autorización	No había una separación clara de las funciones de acceso.	Se creó Usuario.	Se evita mezclar credenciales y permisos con la información laboral de Empleado.

La revisión muestra que la herramienta fue útil para acelerar el diseño inicial, pero no reemplazó el análisis del caso. Las decisiones finales se tomaron revisando las reglas del negocio, las multiplicidades y los principios de POO.

Mejoras aplicadas

Principios de diseño POO

Para revisar el modelo final se consideraron tres principios: cohesión, responsabilidad única y encapsulamiento. La idea fue que cada clase tuviera una función clara y que los datos sensibles no quedaran expuestos.

Clase	Cohesión	Responsabilidad única	Encapsulamiento
Empleado	Se concentra en datos y relaciones del trabajador.	Mantiene información laboral; no gestiona autenticación.	Atributos privados y acceso controlado al salario.
Departamento	Se concentra en la organización del área.	Administra empleados del departamento, no credenciales.	Su lista y datos se modifican mediante métodos.
Proyecto	Se concentra en la iniciativa y sus participantes.	Gestiona empleados y registros asociados al proyecto.	Atributos privados y cambios mediante operaciones.
RegistroTiempo	Se concentra en registrar y validar horas.	No administra empleados ni autenticación.	Valida antes de modificar el registro.
Usuario	Se concentra en el acceso al sistema.	Autentica y autoriza; no administra datos laborales.	ContraseñaHash y rol no se exponen directamente.

Esta revisión también ayuda a mantener el acoplamiento bajo control. Por ejemplo, si cambia la forma de autenticar usuarios, la clase Empleado no debería tener que cambiar por ese motivo. De la misma manera, una modificación en la validación de horas se concentra en RegistroTiempo.

Matriz de trazabilidad

ID	Requerimiento	Clases asociadas	Atributos y métodos
R1	Registrar empleados con identificador único.	Empleado	idEmpleado: int; __init__(): None
R2	Un empleado pertenece a un solo departamento a la vez.	Empleado, Departamento	departamento: Departamento; empleados: list[Empleado]; asignarDepartamento(...): void
R3	Registrar horas con fecha, cantidad, descripción, empleado y proyecto.	RegistroTiempo, Empleado, Proyecto	fecha: date; horas: float; descripcion: str; empleado: Empleado; proyecto: Proyecto; registrar(): void
R4	Proteger la información salarial mediante almacenamiento seguro.	Empleado, Usuario	salarioCifrado: bytes; obtenerSalario(usuario: Usuario): float
R5	Autenticación segura de contraseñas y control de roles.	Usuario	contrasenaHash: str; rol: str; autenticar(...): bool; autorizar(...): bool
R6	Validar que las horas sean mayores a cero y razonables.	RegistroTiempo	horas: float; validarHoras(horas: float): bool
R7	Restringir la eliminación de un departamento si posee empleados.	Departamento	empleados: list[Empleado]; eliminar(): bool
R8	Generar informes de empleados, proyectos, departamentos y registros de tiempo, con posibilidad de exportar a PDF o planilla	Empleado, Departamento, Proyecto, RegistroTiempo	Información registrada en cada clase.

El requisito de generación de informes se relaciona con las clases que contienen la información que debe ser consultada. En esta etapa no se incorpora una clase Informe, ya que la generación y exportación corresponden a una funcionalidad de la aplicación que puede implementarse posteriormente sin modificar las entidades principales del modelo.

La matriz permite comprobar que los requerimientos principales tienen una respuesta concreta en el modelo. También permite detectar si un requisito quedó sin clase, atributo o método relacionado.

IV. Conclusiones

Al desarrollar este trabajo pude pasar desde los problemas del caso de EcoTech Solutions a una estructura de clases más ordenada. La separación entre Empleado, Departamento, Proyecto, RegistroTiempo y Usuario permite que cada parte tenga una función definida y que las relaciones entre ellas se puedan entender directamente en el diagrama.

Uno de los cambios más importantes fue reemplazar la composición entre Departamento y Empleado por una agregación. La decisión se basa en una regla simple del caso: un empleado puede seguir existiendo y ser reasignado aunque cambie el departamento. También fue importante separar las funciones de seguridad en Usuario y validar las horas antes de aceptar un registro.

La revisión con inteligencia artificial fue útil para encontrar alternativas y mejorar el nivel de detalle del modelo, pero también mostró que no conviene aceptar una propuesta automáticamente. En este caso hubo que revisar la relación entre clases, el encapsulamiento y la forma de manejar la seguridad antes de definir el modelo final.

Finalmente, el modelo se puede implementar posteriormente en Python. Para una aplicación real todavía sería necesario agregar persistencia, cifrado real, administración segura de claves, manejo de excepciones y controles de acceso más completos. Por lo tanto, el diagrama representa una base de diseño y no una implementación terminada.

V. Referencias bibliográficas

INACAP. (2026). *Guía de Aprendizaje: Programación Orientada a Objetos Seguros*.
TI3021_U1_ES_GUÍA. Área de Informática y Telecomunicaciones.

INACAP. (2026). *Rúbrica N.º 1: Evaluación Sumativa Unidad 1*. TI3021_U1_RU_ES01_EV02. Sede
Valparaíso.

Larman, C. (2005). *UML y Patrones: Una introducción al análisis y diseño orientado a objetos y al
proceso unificado*. Pearson Educación.

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
Prentice Hall.

Schnettler, R. (2026). *Modelado de una solución en UML: El caso, lo que hay que entregar y la
rúbrica*. Unidad 1, POO Seguro, INACAP Valparaíso.